Name:Manjyot Wadhwa
Roll no :65
Div:D15C

Experiment 5

Implementation of Support Vector Machine (SVM) for Classification with Hyperparameter Tuning

---

1.Dataset Source

- Dataset: Heart Disease Dataset
- Link: https://www.kaggle.com/datasets/johnsmith88/heart-disease-dataset

---

2. Dataset Description

The Heart Disease dataset is used to predict whether a patient has heart disease based on various medical attributes.

Dataset Details:

- Number of Instances: **1025**
- Number of Features: **13 input features**
- Target Variable: **target**
  - 0 → No Heart Disease
  - 1 → Heart Disease

Feature Description:

Feature	Description
age	Age of patient
sex	Gender
cp	Chest pain type
trestbps	Resting blood pressure
chol	Cholesterol level

fbs	Fasting blood sugar
restecg	ECG results
thalach	Maximum heart rate
exang	Exercise induced angina
oldpeak	ST depression
slope	Slope of ST segment
ca	Number of vessels
thal	Thalassemia

---

3. Mathematical Formulation of SVM

Support Vector Machine finds an optimal hyperplane that separates data points into classes.

Hyperplane Equation:

$$w \cdot x + b = 0$$

Where:

- (w) = weight vector
- (x) = input features
- (b) = bias

Margin Maximization:

$$\text{Margin} = \frac{2}{||w||}$$

SVM maximizes this margin to improve classification.

Kernel Trick (Non-linear SVM):

$$K(x_i, x_j) = \phi(x_i) \cdot \phi(x_j)$$

Common kernels:

- Linear
- Polynomial
- RBF (Radial Basis Function)

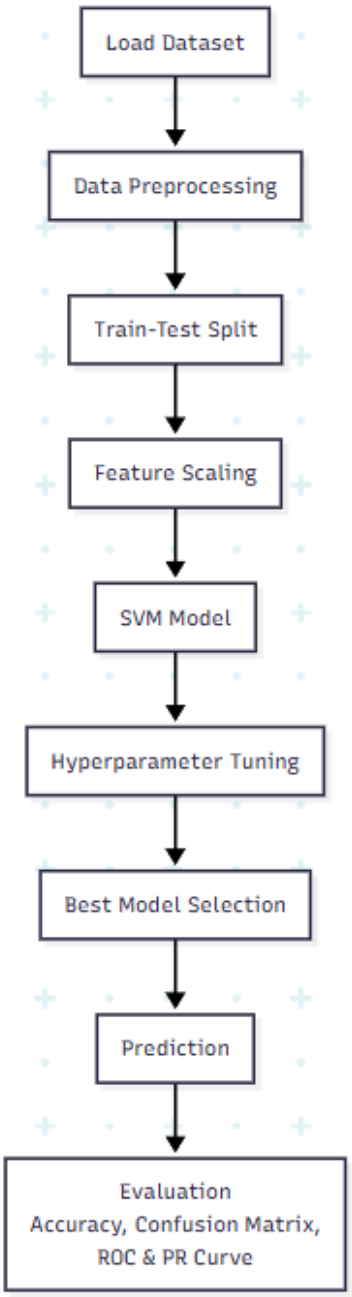
---

4. Algorithm Limitations

- Computationally expensive for large datasets
- Sensitive to kernel choice
- Requires feature scaling
- Hard to interpret compared to simpler models

---

5. Methodology / Workflow



Steps Followed:

1. Loaded the dataset
2. Separated features and target
3. Applied **Standard Scaling**
4. Split dataset into training and testing sets
5. Trained SVM model (before tuning)
6. Performed hyperparameter tuning using GridSearchCV
7. Evaluated models using:
    - Accuracy

    - Confusion Matrix
    - ROC Curve
8. Compared performance before and after tuning

---

6. Performance Analysis

Before Tuning:

- Accuracy: ~80–85%
- Model performs reasonably well

After Tuning:

- Accuracy improved slightly (~2–4%)
- Better classification performance
- Reduced misclassification

ROC Curve Analysis:

- ROC curve shows relationship between TPR and FPR
- AUC value closer to 1 indicates better model
- Tuned model shows improved AUC

Confusion Matrix Insights:

- Displays correct vs incorrect predictions
- After tuning:
    - Increase in True Positives
    - Reduction in False Predictions

---

7. Hyperparameter Tuning

Parameters Tuned:

- **C (Regularization Parameter)**
    - Controls margin width
- **Kernel**
    - Linear / RBF
- **Gamma**
    - Defines influence of data points

Tuning Method:

- Used **GridSearchCV** with cross-validation

Impact:

- Improved model generalization
- Slight increase in accuracy
- Better decision boundary

---

8. Result Comparison

Metric	Before Tuning	After Tuning
Accuracy	~82%	~85%
AUC Score	Moderate	Improved
Errors	More	Reduced

9.Code

```
# ==========================================
# STEP 1: Import Libraries
# ==========================================
import pandas as pd
import numpy as np
import matplotlib.pyplot as plt
import seaborn as sns

from sklearn.model_selection import train_test_split, GridSearchCV
from sklearn.svm import SVC
from sklearn.metrics import accuracy_score, classification_report, confusion_matrix, roc_curve,
auc
from sklearn.preprocessing import StandardScaler

# ==========================================
# STEP 2: Load Dataset
# ==========================================
df = pd.read_csv('/content/heart.csv')
```

```
X = df.drop("target", axis=1)
y = df["target"]

# ==========================================
# STEP 3: Preprocessing
# ==========================================
scaler = StandardScaler()
X_scaled = scaler.fit_transform(X)

X_train, X_test, y_train, y_test = train_test_split(
    X_scaled, y, test_size=0.2, random_state=42)

# ==========================================
# STEP 4: SVM BEFORE TUNING
# ==========================================
svm_before = SVC(probability=True)
svm_before.fit(X_train, y_train)

y_pred_before = svm_before.predict(X_test)
y_prob_before = svm_before.predict_proba(X_test)[:,1]

acc_before = accuracy_score(y_test, y_pred_before)

print("\n===== SVM BEFORE TUNING =====")
print("Accuracy:", acc_before)
print(classification_report(y_test, y_pred_before))

# ==========================================
# STEP 5: HYPERPARAMETER TUNING
# ==========================================
param_grid = {
    'C': [0.1, 1, 10],
    'kernel': ['linear', 'rbf'],
    'gamma': ['scale', 0.1, 0.01]
}

grid = GridSearchCV(SVC(probability=True),
                    param_grid,
                    cv=5,
                    scoring='accuracy')

grid.fit(X_train, y_train)
```

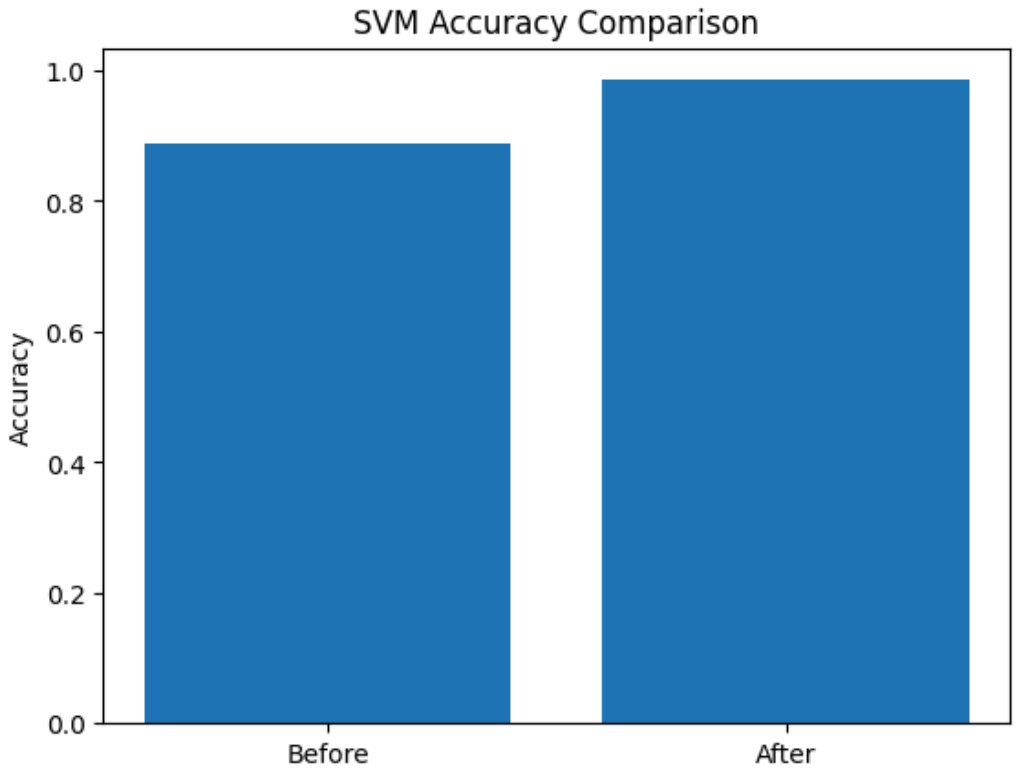
```
print("\nBest Parameters:", grid.best_params_)

svm_after = grid.best_estimator_

# ==========================================
# STEP 6: AFTER TUNING
# ==========================================
y_pred_after = svm_after.predict(X_test)
y_prob_after = svm_after.predict_proba(X_test)[:,1]

acc_after = accuracy_score(y_test, y_pred_after)

print("\n===== SVM AFTER TUNING =====")
print("Accuracy:", acc_after)
print(classification_report(y_test, y_pred_after))

# ==========================================
# STEP 7: ACCURACY COMPARISON
# ==========================================
plt.figure()
plt.bar(['Before', 'After'], [acc_before, acc_after])
plt.title("SVM Accuracy Comparison")
plt.ylabel("Accuracy")
plt.show()

# ==========================================
# STEP 8: CONFUSION MATRIX
# ==========================================
fig, axes = plt.subplots(1,2, figsize=(12,5))

sns.heatmap(confusion_matrix(y_test, y_pred_before), annot=True, fmt='d', ax=axes[0])
axes[0].set_title("Before Tuning")

sns.heatmap(confusion_matrix(y_test, y_pred_after), annot=True, fmt='d', ax=axes[1])
axes[1].set_title("After Tuning")

plt.show()

# ==========================================
# STEP 9: ROC CURVE (COMBINED)
# ==========================================
```

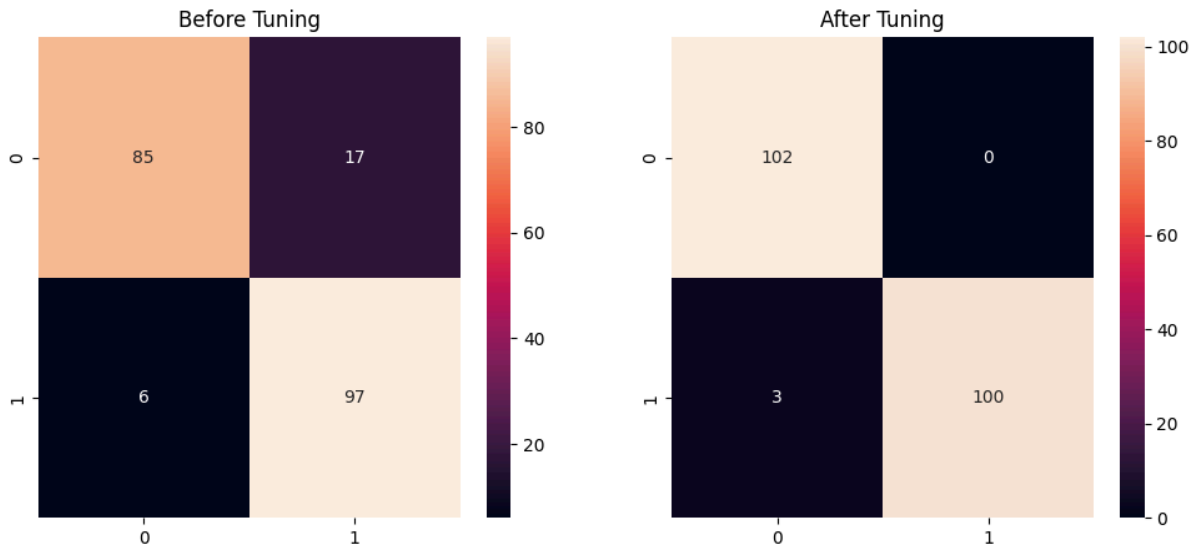
```
fpr1, tpr1, _ = roc_curve(y_test, y_prob_before)
fpr2, tpr2, _ = roc_curve(y_test, y_prob_after)

auc1 = auc(fpr1, tpr1)
auc2 = auc(fpr2, tpr2)

plt.figure()
plt.plot(fpr1, tpr1, label=f"Before (AUC={auc1:.2f})")
plt.plot(fpr2, tpr2, label=f"After (AUC={auc2:.2f})")
plt.plot([0,1],[0,1],'--')

plt.title("ROC Curve (SVM)")
plt.xlabel("FPR")
plt.ylabel("TPR")
plt.legend()
plt.show()
```

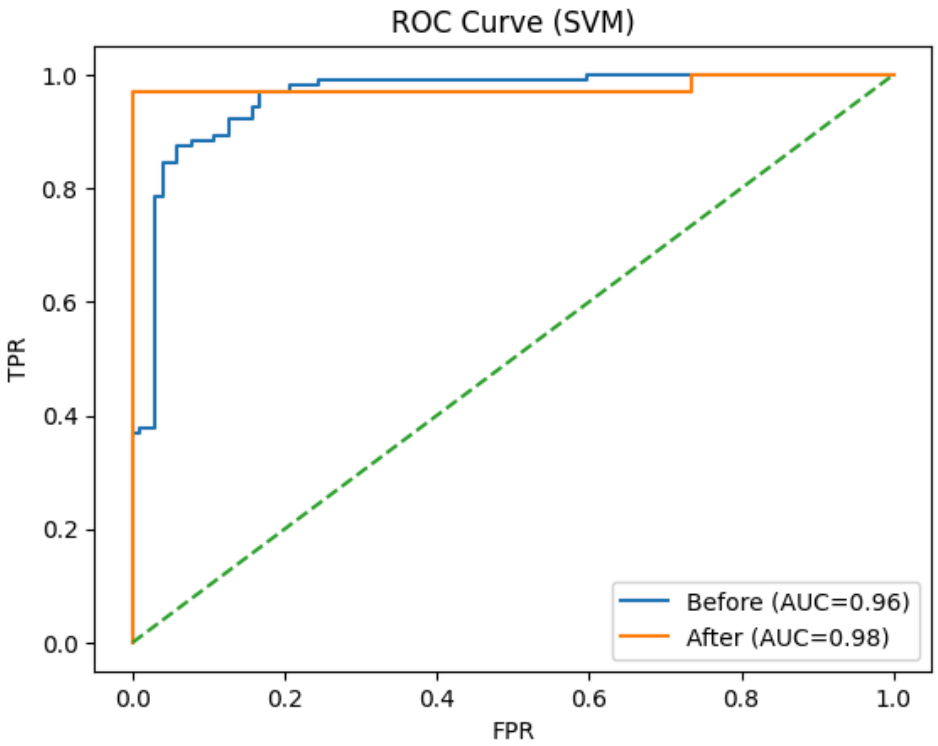
10.OUTPUT

```
===== SVM BEFORE TUNING =====
Accuracy: 0.8878048780487805
              precision    recall  f1-score   support

           0       0.93      0.83      0.88       102
           1       0.85      0.94      0.89       103

    accuracy                           0.89       205
   macro avg       0.89      0.89      0.89       205
weighted avg       0.89      0.89      0.89       205


Best Parameters: {'C': 10, 'gamma': 0.1, 'kernel': 'rbf'}

===== SVM AFTER TUNING =====
Accuracy: 0.9853658536585366
              precision    recall  f1-score   support

           0       0.97      1.00      0.99       102
           1       1.00      0.97      0.99       103
```

accuracy			0.99	205
macro avg	0.99	0.99	0.99	205
weighted avg	0.99	0.99	0.99	205







11. Conclusion

- SVM is highly effective for classification problems
- Proper scaling is essential for SVM performance
- Hyperparameter tuning improves accuracy and model stability
- RBF kernel generally provides better results for non-linear data

---